The scanf() Function in C

The scanf() function is a standard input function in C. It is used to read data entered by the user from the keyboard and store that data in variables.

It is declared in the header file stdio.h.

1. **Basic Syntax of scanf()**

- `scanf`: The function that reads input.
- `format_specifier`: Specifies the type of data to read.
- `&variable`: Gives the memory address where the input value should be stored.

```
#include <stdio.h>
int main(void)
{
   int age;
   printf("Enter your age: ");
   scanf("%d", &age);
   printf("Your age is %d\n", age);
   return 0;
}
```

2. Format Specifiers

The format specifier tells `scanf()` what type of input to expect.

Data type	Variable example	`scanf()` specifier	Example input
`int`	`int age;`	`%d`	`25`
`float`	`float price;`	`%f`	`12.5`
`double`	`double value;`	`%lf`	`3.14159`
`char`	`char letter;`	`%c`	`A`
String	`char name[30];`	`%29s`	`Alice`
`short`	`short n;`	`%hd`	`100`
`long`	`long n;`	`%ld`	`123456`
`long long`	`long long n;`	`%lld`	`12345678900`

For `scanf()`, use `%f` for a `float` and `%lf` for a `double`. This differs from `printf()`, where `%f` is used to print both `float` and `double` arguments.

3. **Reading Different Data Types**

Example 2: Reading an integer and a float

```
#include <stdio.h>
int main(void)
{
   int quantity;
   float price;

   printf("Enter quantity: ");
   scanf("%d", &quantity);
   printf("Enter price: ");
   scanf("%f", &price);
   printf("Quantity = %d\n", quantity);
   printf("Price = %.2f\n", price);
   return 0;
}
```

The format specifier %.2f belongs to printf() and displays two digits after the decimal point.

Example 3: Reading a double

```
#include <stdio.h>
int main(void)
{
   double pi;
   printf("Enter the value of pi: ");
   scanf("%lf", &pi);
   printf("Value = %.5f\n", pi);
   return 0;
}
```

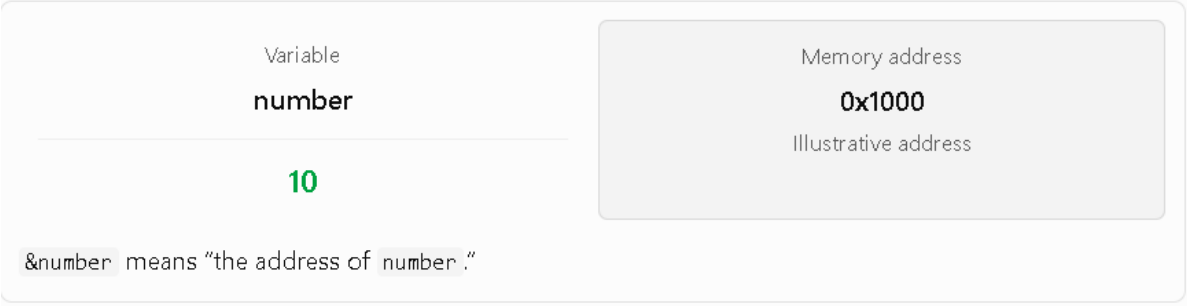
4. **Why Do We Use & in scanf()?**

The & operator is called the address-of operator. It obtains the memory address of a variable.

Consider:

```
int number = 10;
```

Conceptually, memory might look like this:



When you write:

```
scanf("%d", &number);
```

ou give scanf() the address of the variable, allowing it to store the input at that location.
Incorrect:

```
int number;
scanf("%d", number);  // Undefined behavior
```

Here, number is an uninitialized integer, not a valid pointer to the variable's memory.
Passing it this way causes undefined behavior.
Correct:

```
int number;
scanf("%d", &number);
```

5. **Reading Multiple Variables in One scanf()**

You can read multiple values using one call to scanf().
Example : Reading two integers

```
#include <stdio.h>
int main(void)
{
   int a, b;
   printf("Enter two integers: ");
   scanf("%d %d", &a, &b);
   printf("a = %d, b = %d\n", a, b);
   printf("Sum = %d\n", a + b);
   return 0;
}
```

The input can also be entered on separate lines:

```
Enter two integers: 10
20
```

Whitespace in the format string, including spaces and newlines, allows scanf() to skip whitespace when reading most numeric values.
Example: Reading three different types

```
#include <stdio.h>
int main(void)
{
   int age;
   float height;
   char grade;
   printf("Enter age, height, and grade: ");
   scanf("%d %f %c", &age, &height, &grade);
   printf("Age: %d\n", age);
   printf("Height: %.2f\n", height);
   printf("Grade: %c\n", grade);
   return 0;
}
```

The variables must appear in the same order as their corresponding format specifiers.

6. The Important Difference Between %c and " %c"

This is a common source of bugs for beginners.
Consider the following code:

```
int age;
char grade;

scanf("%d", &age);
scanf("%c", &grade);
```

Suppose the user enters

```
20
A
```

After reading 20, the newline character generated by pressing Enter may remain in the input stream. The next %c reads that newline instead of A.
Solution: Put a space before %c.

```
scanf("%d", &age);
scanf(" %c", &grade);
```

The leading space tells scanf() to skip any whitespace before reading the character.
Example:

```
#include <stdio.h>
int main(void)
{
   int age;
   char grade;
   printf("Enter age: ");
   scanf("%d", &age);
   printf("Enter grade: ");
   scanf(" %c", &grade);
   printf("Age = %d, Grade = %c\n", age, grade);
   return 0;
}
```

Remember that %c reads one character, including whitespace. The leading space is useful when you want to skip whitespace before reading a character.

7. Common Mistakes in scanf()

Mistake	Incorrect code	Correct approach
Forgetting &	scanf("%d", n);	scanf("%d", &n);
Using %f for a double	scanf("%f", &d);	scanf("%lf", &d);
Reading a character after numeric input	scanf("%c", &ch);	scanf(" %c", &ch); when whitespace should be skipped

Operators in C: Arithmetic, Comparison, and Logical Operators

In the C programming language, operators are symbols used to perform operations on variables and values. They are essential for calculations, making decisions, and controlling the flow of a program.

This lecture covers three important groups of operators:

1. Arithmetic operators
2. Comparison (relational) operators
3. Logical operators

All examples are written in C and include explanations of their outputs.

1. Arithmetic Operators

Arithmetic operators perform mathematical calculations on numeric values.

Operator	Name	Example	Result
+	Addition	10 + 3	13
-	Subtraction	10 - 3	7
*	Multiplication	10 * 3	30
/	Division	10 / 3	3
%	Modulus (remainder)	10 % 3	1
++	Increment	x++	Increases x by 1
--	Decrement	x--	Decreases x by 1

1.1 Addition, subtraction, and multiplication

```
#include <stdio.h>
int main(void)
{
   int a = 10, b = 3;
   printf("Addition: %d\n", a + b);
   printf("Subtraction: %d\n", a - b);
   printf("Multiplication: %d\n", a * b);
   return 0;
}
```

Explanation:

- a + b adds the two values.
- a - b subtracts b from a.
- a * b multiplies the two values.

1.2 Division: Integer vs. floating-point division

One of the most important rules in C is that integer division discards the fractional part when both operands are integers.

```
#include <stdio.h>
int main(void)
{
   int a = 10, b = 3;
   printf("%d\n", a / b);
   printf("%.2f\n", (double)a / b);
   return 0;
}
```

Why?

- 10 / 3 performs integer division and produces 3.
- (double)a / b converts a to a floating-point value before division, producing approximately 3.3333.
- %.2f displays two digits after the decimal point.

```
printf("%d\n", 7 / 2);      // 3
printf("%.1f\n", 7.0 / 2);   // 3.5
```

1.3 The modulus operator %

The modulus operator returns the remainder of integer division.

```
printf("%d\n", 10 % 3);  // 1
printf("%d\n", 14 % 5);  // 4
printf("%d\n", 20 % 4);  // 0
printf("%d\n", 17 % 2);  // 1
```

1.4 Increment and decrement operators

The increment operator ++ increases a variable by one. The decrement operator -- decreases it by one.

```
int x = 5;
x++;
printf("%d\n", x);  // 6
x--;
printf("%d\n", x);  // 5
```

Prefix vs. postfix

The distinction matters when the operator is used inside a larger expression.

- Prefix: ++x — increment first, then use the new value.
- Postfix: x++ — use the old value, then increment.

```
#include <stdio.h>
int main(void)
{
   int x = 5;
   int a, b;
   a = ++x;
   printf("a = %d, x = %d\n", a, x);
   x = 5;
   b = x++;
   printf("b = %d, x = %d\n", b, x);
   return 0;
}
```

In the first case, x is incremented before its value is assigned to a. In the second case, the original value of x is assigned to b before x is incremented.

Avoid expressions such as printf("%d %d", x++, ++x);. Modifying the same variable more than once without a properly defined sequencing relationship can cause undefined behavior in C.

2. Comparison (Relational) Operators

Comparison operators compare two values. They are commonly used in if statements, loops, and other decision-making structures.

In C, a comparison produces an integer result:

- 1 means true.
- 0 means false.

Operator	Meaning	Example	Result
==	Equal to	5 == 5	1
!=	Not equal to	5 != 3	1
>	Greater than	8 > 4	1
<	Less than	2 < 7	1
>=	Greater than or equal to	5 >= 5	1
<=	Less than or equal to	3 <= 2	0

2.1 Examples of comparison operators

```
#include <stdio.h>
int main(void)
{
   int a = 10, b = 5;
   printf("%d\n", a == b);
   printf("%d\n", a != b);
   printf("%d\n", a > b);
   printf("%d\n", a < b);
   printf("%d\n", a >= 10);
   printf("%d\n", b <= 4);
```

```
    return 0;
}
```

Let's examine each expression:

- a == b: false, because 10 is not equal to 5.
- a != b: true, because 10 differs from 5.
- a > b: true, because 10 is greater than 5.
- a < b: false, because 10 is not less than 5.
- a >= 10: true, because 10 is equal to 10.
- b <= 4: false, because 5 is greater than 4.

2.2 The difference between = and ==

This is one of the most common mistakes in C programming.

- = is the assignment operator.
- == is the equality comparison operator.

```
int x = 5;  // Assign 5 to x

printf("%d\n", x == 5);  // 1: comparison is true
printf("%d\n", x == 8);  // 0: comparison is false
```

3. Logical Operators

Logical operators combine conditions or reverse the truth value of a condition. They are particularly important when a program must make decisions based on multiple requirements.

C has three fundamental logical operators:

Operator	Name	Meaning
&&	Logical AND	True if both conditions are true
||	Logical OR	True if at least one condition is true
!	Logical NOT	Reverses the truth value

In C, zero represents false, and any nonzero value represents true. Logical operators produce either 0 or 1.

3.1 Logical AND (&&)

The AND operator returns true only when both conditions are true.

Truth table:

Condition A	Condition B	A && B
0	0	0
0	1	0
1	0	0
1	1	1

Example:

```
#include <stdio.h>
int main(void)
{
   int age = 25;
   int hasLicense = 1;
   printf("%d\n", age >= 18 && hasLicense == 1);
   printf("%d\n", age < 18 && hasLicense == 1);
   printf("%d\n", age >= 18 && hasLicense == 0);
   return 0;
}
```

Explanation:

1. age >= 18 is true, and hasLicense == 1 is true. Therefore, the result is 1.
2. age < 18 is false. AND returns 0 even though the second condition is true.
3. The first condition is true, but the second is false. The result is 0.

3.2 Logical OR (||)

The OR operator returns true if at least one condition is true. It returns false only when both conditions are false.

Truth table:

Condition A	Condition B	A || B
0	0	0
0	1	1
1	0	1
1	1	1

3.3 Logical NOT (!)

The NOT operator reverses the truth value of an expression.

Condition A	!A
0	1
1	0

Example:

```
#include <stdio.h>
int main(void)
{
   int isRaining = 0;
   printz("%d\n", !isRaining);
   printf("%d\n", !!isRaining);
   return 0;
}
```

Explanation:

- !isRaining is true because isRaining is zero.

- !!isRaining reverses the value twice, so the result is false.

4. Combining Comparison and Logical Operators
In real programs, logical operators are often used to combine comparison expressions

5. Short-Circuit Evaluation

This is an important feature of C logical operators.

- With `A && B`, if `A` is false, C does not need to evaluate `B`.
- With `A || B`, if `A` is true, C does not need to evaluate `B`.

This behavior is called short-circuit evaluation.